

Graduation Project Proposal

Government Services Performance & Complaints ETL Platform

A Scalable, Cloud-Native Data Engineering Solution for Citizen Complaint and Service-Quality Analytics

Prepared by: [Osama Abdulhamid]

Program: Data Science — Digital Egypt Pioneers Initiative (DEPI)

Track: Big Data Engineering

Supervisor: [Ahmed Azab]

Table of Contents

1. Executive Summary	3
2. Introduction and Background	3
3. Problem Statement.....	3
4. Project Objectives	4
4.1 Primary Objectives.....	4
4.2 Secondary (Stretch) Objectives	4
5. Project Scope	4
5.1 In Scope	4
5.2 Out of Scope.....	5
5.3 Assumptions	5
6. Proposed Solution and Methodology.....	5
7. System Architecture	5
8. Technology Stack.....	6
9. Data Model Overview	7
9.1 Fact Tables	7
9.2 Dimension Tables	7
10. Key Performance Indicators	8
10.1 Platform / Pipeline KPIs	8
10.2 Business KPIs (Dashboard).....	8
11. Project Plan and Timeline	8
12. Risk Assessment and Mitigation.....	9
13. Expected Deliverables.....	10
14. Future Enhancements	10
15. Conclusion	11
16. References.....	11

1. Executive Summary

Government entities generate and collect vast amounts of data relating to citizen complaints, service ratings, and departmental performance. In practice, however, this data is fragmented across disparate sources and formats — CSV and JSON files, relational databases, and external APIs — which makes it difficult for decision-makers to obtain a unified and timely view of service performance.

This project proposes the design and implementation of a scalable, cloud-native ETL (Extract, Transform, Load) platform built on Microsoft Azure that consolidates these heterogeneous sources into a centralised data warehouse and surfaces actionable insights through an executive dashboard. The platform automates the end-to-end data lifecycle using Apache Airflow and Azure Data Factory, applies a modern Medallion (Bronze–Silver–Gold) architecture for progressive data refinement, and delivers interactive Power BI dashboards that empower senior management to monitor complaint-resolution performance, citizen satisfaction, and operational bottlenecks in near real time.

2. Introduction and Background

As part of broader national digital-transformation efforts, government service providers increasingly depend on data to improve the quality and responsiveness of public services. Citizens interact with these services through multiple channels — web portals, mobile applications, call centres, and in-person visits — each generating valuable feedback in the form of complaints and service ratings. When aggregated and analysed effectively, this feedback can reveal systemic issues, distinguish high-performing from underperforming departments, and guide resource-allocation decisions.

Yet the value of this data is rarely realised. It is scattered across incompatible systems, stored in inconsistent formats, and lacks the standardisation required for meaningful analysis. Manual consolidation is slow, error-prone, and unsustainable at scale. A robust data-engineering solution is therefore needed to automate the collection, cleansing, integration, and presentation of this information. This project addresses that need by building a production-grade data pipeline that demonstrates industry-standard cloud data-engineering practices while solving a concrete, socially relevant problem.

3. Problem Statement

The central problem this project addresses can be summarised as follows: government service data is fragmented, unstandardised, and disconnected from decision-making. The specific challenges are:

- Data originates from multiple heterogeneous sources (citizen complaints, service-rating systems, governorate records, and ministry records) in varied formats such as CSV, JSON, relational tables, and API responses.
- There is no centralised repository that integrates these sources into a single, consistent view.
- The absence of automated processing means data is not refreshed reliably, and quality issues — duplicates, inconsistent naming, and missing values — remain unaddressed.
- Decision-makers lack timely, trustworthy analytics on key questions such as complaint-resolution speed, departmental performance, and citizen satisfaction.

As a result, valuable operational intelligence remains locked in silos, and leadership decisions are made without a complete, data-driven picture.

4. Project Objectives

4.1 Primary Objectives

1. Design and implement an automated, scalable ETL pipeline that ingests data from multiple heterogeneous sources.
2. Orchestrate the end-to-end workflow using Apache Airflow integrated with Azure Data Factory, supporting both batch and event-driven ingestion.
3. Apply a Medallion (Bronze–Silver–Gold) architecture to progressively cleanse, standardise, and enrich the data.
4. Model and populate a centralised data warehouse in Azure SQL Database using a dimensional (star-schema) design optimised for analytical queries.
5. Implement data-quality validation and pipeline monitoring with automated alerting on failures and performance anomalies.
6. Develop an interactive executive dashboard in Power BI that presents key performance indicators for senior management.

4.2 Secondary (Stretch) Objectives

- Integrate a machine-learning model for anomaly detection, providing early warning of unusual spikes in complaints for a given service or region.
- Provide a natural-language query interface (text-to-SQL) using Azure OpenAI Service, allowing non-technical managers to query the data conversationally.
- Apply Arabic natural-language processing to automatically categorise and gauge sentiment from free-text complaint descriptions.

5. Project Scope

5.1 In Scope

- The full ETL pipeline: extraction from sources, transformation, and loading into the warehouse.
- Workflow orchestration, scheduling, and monitoring.
- The centralised data warehouse and its dimensional (star-schema) model.
- The interactive executive dashboard.
- Automated data-quality checks and validation.
- Complete project documentation and deliverables in line with the project guidelines.

5.2 Out of Scope

- Development of a public-facing complaint-submission application.
- Integration with live, real government systems; the project uses representative synthetic data.
- Long-term production deployment and operational maintenance beyond the academic timeframe.

5.3 Assumptions

- Realistic synthetic data will be generated to emulate the described sources.
- Azure resources will be available through a provided or student subscription.
- The project is designed and executed individually.

6. Proposed Solution and Methodology

The proposed solution is a cloud-native data platform implemented on Microsoft Azure, following the Extract–Transform–Load paradigm and structured around the Medallion architecture — an industry-standard pattern for organising a data lake into progressive layers of quality and readiness.

Methodologically, the project adopts an iterative, milestone-driven approach. A minimal end-to-end pipeline (a “walking skeleton”) is built first to validate the architecture, after which capabilities — additional sources, event-driven streaming, data-quality automation, and analytical enhancements — are layered on incrementally. This de-risks the project by ensuring a working deliverable exists at every stage, and it allows the most valuable features to be prioritised should time become constrained.

7. System Architecture

The platform is organised into the following logical layers, each implemented with managed Azure services and coordinated by Apache Airflow:

- **Data Sources Layer:** citizen complaints, service ratings, governorate data, and ministry data, supplied as CSV/JSON files, database extracts, and API responses.
- **Ingestion Layer:** Azure Data Factory copy activities ingest batch sources, while Azure Event Hubs captures event-driven complaint streams; Apache Airflow schedules and triggers these workflows.
- **Data Lake (Storage) Layer:** Azure Data Lake Storage Gen2, organised into Bronze (raw), Silver (cleansed), and Gold (curated) zones following the Medallion pattern.
- **Transformation Layer:** Azure Data Factory Mapping Data Flows, executed on a managed Spark runtime, perform cleansing, deduplication, standardisation, and aggregation.
- **Data Warehouse Layer:** Azure SQL Database hosts the curated star-schema model that powers analytics.
- **Serving and Presentation Layer:** Microsoft Power BI connects to the warehouse to deliver interactive executive dashboards.
- **Orchestration and Monitoring:** Apache Airflow coordinates the pipeline end to end, triggering Azure Data Factory pipelines through its REST API, with logging, automatic retries, and failure alerts.

Detailed visual artefacts — the system architecture diagram, entity-relationship diagram (ERD), data-flow diagrams (DFD), and component and deployment diagrams — will be produced during the Analysis and Design phase, in accordance with the project documentation guidelines.

8. Technology Stack

The platform is built on a Microsoft Azure technology stack that reflects current industry practice in cloud data engineering.

Layer / Function	Technology	Purpose
Ingestion (batch)	Azure Data Factory	Copy data from files, databases, and APIs into the data lake.
Ingestion (streaming)	Azure Event Hubs	Capture event-driven complaint data in near real time (Kafka-compatible).
Data lake storage	Azure Data Lake Storage Gen2	Store raw, cleansed, and curated data (Bronze / Silver / Gold).
Transformation	ADF Mapping Data Flows	Cleanse, standardise, and aggregate data at scale on a managed Spark runtime.

Layer / Function	Technology	Purpose
Data warehouse	Azure SQL Database	Host the curated dimensional (star-schema) model.
Orchestration	Apache Airflow	Schedule, coordinate, and trigger pipeline workflows.
Monitoring & alerting	Airflow + Azure Monitor	Track pipeline health, log runs, and alert on failures.
Visualisation	Microsoft Power BI	Deliver interactive executive dashboards.
Scripting & logic	Python	Synthetic-data generation, custom logic, and Airflow DAGs.
Version control	Git & GitHub	Source-code management and version history.
NL query (stretch)	Azure OpenAI Service	Natural-language (text-to-SQL) querying for managers.
ML / anomaly (stretch)	Azure Machine Learning	Detect anomalous complaint patterns.

9. Data Model Overview

The data warehouse adopts a dimensional star-schema design, optimised for fast analytical queries and intuitive reporting. Fact tables capture measurable business events, while dimension tables provide the descriptive context used to slice and filter those events.

9.1 Fact Tables

- **fact_complaints** — one row per complaint, with measures such as resolution time, resolution status, and reopen count.
- **fact_service_ratings** — one row per rating, with measures such as rating score and waiting time.

9.2 Dimension Tables

- **dim_date** — calendar attributes for time-based analysis.
- **dim_governorate** — geographic attributes (governorate and region).
- **dim_ministry** — the responsible ministry or authority.
- **dim_service** — the specific government service and its category.
- **dim_channel** — the submission channel (web, mobile, call centre, in person).
- **dim_status** — the lifecycle status of a complaint.

Within the data lake, the Medallion architecture governs progressive refinement: the Bronze layer stores raw ingested data as received; the Silver layer holds cleansed, deduplicated, and standardised records; and the Gold layer contains the curated, aggregated data that populates the warehouse.

10. Key Performance Indicators

Two categories of KPI are defined: those measuring the success of the platform itself, and the business indicators the dashboard surfaces for decision-makers.

10.1 Platform / Pipeline KPIs

- Pipeline reliability — successful end-to-end runs (target of at least 95 per cent).
- Data freshness — elapsed time between a source update and its availability in the warehouse.
- Data quality — percentage of records passing automated validation checks.
- Processing performance — pipeline execution time kept within defined thresholds.

10.2 Business KPIs (Dashboard)

Indicator	What it measures
Total complaints over time	The volume and trend of complaints across the reporting period.
Complaint resolution rate	The proportion of complaints successfully resolved.
Average resolution time & SLA compliance	How quickly complaints are resolved and whether targets are met.
Complaints by governorate	The geographic distribution of complaints across regions.
Complaints by ministry / service	Which departments and services attract the most complaints.
Average citizen-satisfaction score	The mean service rating provided by citizens.
Top recurring complaint categories	The most frequent complaint themes requiring attention.

11. Project Plan and Timeline

The project follows the phased structure defined in the programme guidelines, mapped to the academic schedule below. An iterative methodology is applied within the

Implementation phase, delivering a minimal working pipeline first and adding capabilities incrementally.

Phase	Timeframe	Key Activities	Deliverables
1. Planning & Proposal	Week 5	Define the problem, objectives, scope, and architecture; prepare the proposal.	Project proposal and plan.
2. Instructor Review	Week 7	Present the proposal and incorporate supervisor feedback.	Approved, revised proposal.
3. Requirements Gathering	Weeks 9–10	Identify stakeholders, use cases, and functional / non-functional requirements.	Requirements specification.
4. Analysis & Design	Weeks 13–15	Produce the architecture diagram, ERD, DFD, data model, and dashboard wireframes.	Design documentation and diagrams.
5. Implementation	Weeks 16–21	Build the pipeline (ingestion, transformation, warehouse, dashboard), orchestration, data quality, streaming, and stretch features.	Working platform and source code.
6. Testing & QA	Weeks 22–23	Test the pipeline, validate data, and document and resolve issues.	Test plan, test cases, bug reports.
7. Final Presentation	Week 24	Finalise documentation; prepare the presentation and demonstration.	Final report, presentation, demo.

12. Risk Assessment and Mitigation

The principal risks to the project and the measures planned to mitigate them are set out below.

Risk	Impact	Likelihood	Mitigation
Azure costs exceeding the available budget	High	Medium	Use the provided / student subscription and free tiers; monitor spend with Azure Cost Management; shut down idle resources.

Risk	Impact	Likelihood	Mitigation
Learning curve on Azure Data Factory and Airflow	Medium	High	Allocate early learning time; follow official documentation; build incrementally.
Data-quality issues in source data	Medium	Medium	Implement automated validation; design realistic data-generation logic.
Scope creep beyond core requirements	Medium	Medium	Prioritise an MVP; treat ML and NL features as optional stretch goals.
Live-demonstration failure during the defence	High	Low	Prepare a pre-recorded backup demo; rehearse; keep a stable dataset.
Workload constraints of a solo project	Medium	Medium	Follow a phased plan with buffer time; complete core deliverables before enhancements.
Unavailability of real government data	Low	High	Generate representative synthetic data mirroring realistic structures and distributions.

13. Expected Deliverables

- A fully functional, automated ETL pipeline on Microsoft Azure.
- A populated data warehouse implementing a star-schema dimensional model.
- An interactive Power BI executive dashboard.
- End-to-end orchestration (Airflow + Azure Data Factory) with monitoring and alerting.
- Complete project documentation: requirements, design diagrams, technical documentation, and a user manual.
- Source code hosted on GitHub with a clear commit history.
- A final presentation and a working demonstration of the platform.

14. Future Enhancements

Beyond the core deliverables, the platform is designed to accommodate the following enhancements, which also serve as stretch objectives for this project:

- **Anomaly detection:** a machine-learning model that provides early warning of abnormal spikes in complaints for a particular service or region, enabling proactive intervention.

- **Conversational analytics:** a natural-language interface (text-to-SQL) powered by Azure OpenAI Service, allowing managers to query the data in plain language.
- **Arabic text analytics:** automated categorisation and sentiment analysis of free-text complaint descriptions using Arabic NLP models.

15. Conclusion

This project delivers a practical, end-to-end demonstration of modern cloud data engineering applied to a meaningful public-sector problem. By transforming fragmented government service data into a unified, automated, and insight-rich platform, it bridges the gap between raw operational data and informed decision-making. The Azure technology stack chosen reflects current industry practice, ensuring that the skills demonstrated are directly relevant to professional data-engineering roles.

16. References

- Microsoft. Azure Data Factory documentation. Microsoft Learn.
- Microsoft. Azure Data Lake Storage and the Medallion lakehouse architecture. Microsoft Learn.
- Apache Software Foundation. Apache Airflow documentation.
- Kimball, R. and Ross, M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. Wiley.